
CONTEXTUAL LEARNING FOR INTERRUPTION TOLERANCE AND MULTITASKING

Thomas E. Portegys, portegys@gmail.com, ORCID 0000-0003-0087-6363

Dialectek, DeKalb, Illinois USA

ABSTRACT

This project is an effort to combine the Morphognosis and Mona neural network models into a comprehensive model for learning and behavior called Mandala. Mona features a contextual causation learning with goal-directed motivation. Morphognosis features contextual multilayer perceptron (MLP) learning. Mandala achieves this by accumulating contextual tiers of temporal states that are fed into an MLP at each time step. Natural environments abound in event streams that require interruption tolerance and multitasking. Mandala affords multitasking as it is robust in the presence of intervening events from overlaid causation chains, a capability that conventional recurrent artificial neural networks (RNNs) struggle with. The results bear this out. In addition, accumulating temporal information discretely labels hierarchical cause-and-effect relationships that can be used for augmented processing. In the case of Mona, channeling motivation through the network for the purpose of goal-seeking requires this feature.

Keywords: Mandala, causation learning, multitasking, artificial neural network, machine learning.

INTRODUCTION

A mated pair of eagles, Jackie and Shadow, are popular on the internet these days as viewed with a hidden camera and microphone (Forgione, 2020). They spend their days and nights building and repairing their nest, brooding eggs, hunting, and fending off threats. In one segment Shadow is seen screaming for Jackie to help with intruding ravens that have in the past destroyed their eggs. He knows he must remain sitting atop them. She has been gone overlong and he must be hungry. This in general is a typical condition for animals and the neural networks that orchestrate them. Multitasking in the midst of interruptions and interferences between goals.

Artificial neural networks (ANNs) currently deal with what might be termed controlled surprise. Self-driving cars are in the focused business of driving. They can handle unexpected road obstructions which they have previously seen in training. Large Language Models (LLMs) (Yin, et al., 2024) can sensibly respond to just about any query, but while composing said response they do not deal with a crying baby or a flat tire. Human animals do.

This project is an effort to find a way how interruptions can be accommodated and multitasking can be accomplished more effectively by an ANN. The resulting architecture is named Mandala, after a geometric pattern that is typically composed of concentric rings or tiers of symbols having some comprehensive meaning (Merriam–Webster Online Dictionary, 2008). Here Mandala is also a tiered construction, where higher tiers represent longer-term context that aids in prediction.

Mandala is also a step toward synthesizing the Morphognosis (Portegys, 2022) and Mona (Portegys, 2010 and 2024b) neural network models into a comprehensive model for learning and behavior. Mona features a contextual causation learning with goal-directed motivation. Morphognosis features contextual multilayer perceptron (MLP) learning. Mandala achieves this by accumulating contextual tiers of temporal states that are fed into an MLP at each time step. In addition, accumulating non-hidden temporal information discretely labels hierarchical cause-and-effect relationships that can be used for augmented processing. In the case of Mona, channeling motivation through the network for the purpose of goal-seeking requires this feature.

Architecturally Mandala bears a resemblance to a Jordan recurrent neural network (Jordan, 1986), as shown in Figure 1.

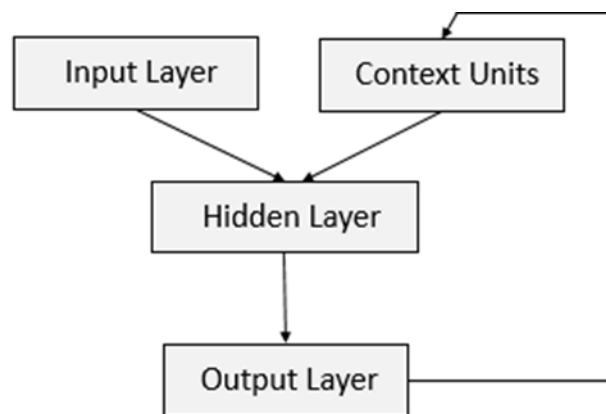


Figure 1 – Jordan recurrent neural network

In a Jordan network, the output layer is fed into context units, which in turn are fed into the hidden layer along with the input layer. In Mandala, the output layer produces not only a prediction, but also temporal state, which after processing combines with external input.

Mandala also shares some commonalities with networks that make use functionality external to the core network. Some of these are discussed below.

Recurrent Neural Networks (RNNs) with external temporal memory are architectures designed to overcome the limitations of standard RNNs (like LSTMs or GRUs) in memorizing long-term dependencies. While standard RNNs rely on internal hidden states that often suffer from

vanishing gradients over long sequences, these models integrate a separate, addressable memory bank to store and retrieve past information over extended time horizons.

Neural Turing Machines (NTMs) (Graves et al., 2014). This is a model that couples a neural network controller to an external memory bank, capable of learning to manipulate and manage the memory to solve tasks like sorting or copying.

A Dynamic Memory Network (DMN) (Kumar et al., 2016) is a neural network architecture that uses an external episodic memory to answer questions about input sequences (stories, text).

External Addressable Long-Term and Working Memory (EALWM) (Quan et al., 2020) splits external memory into distinct long-term and working memory parts, improving the ability to learn long-term dependencies without gradient issues.

The above models retain an RNN at their core, using external storage as an adjunct memory. In Mandala external memory serves as the sole repository of temporal state, combining environment and state information as input to a non-recurrent MLP.

DESCRIPTION

PROBLEM GENERATION

Training and testing event streams are generated using randomly generated grammars that produce binary hierarchies of cause-and-effect pairs of terminals. A simple such hierarchy is presented in Figure 2.

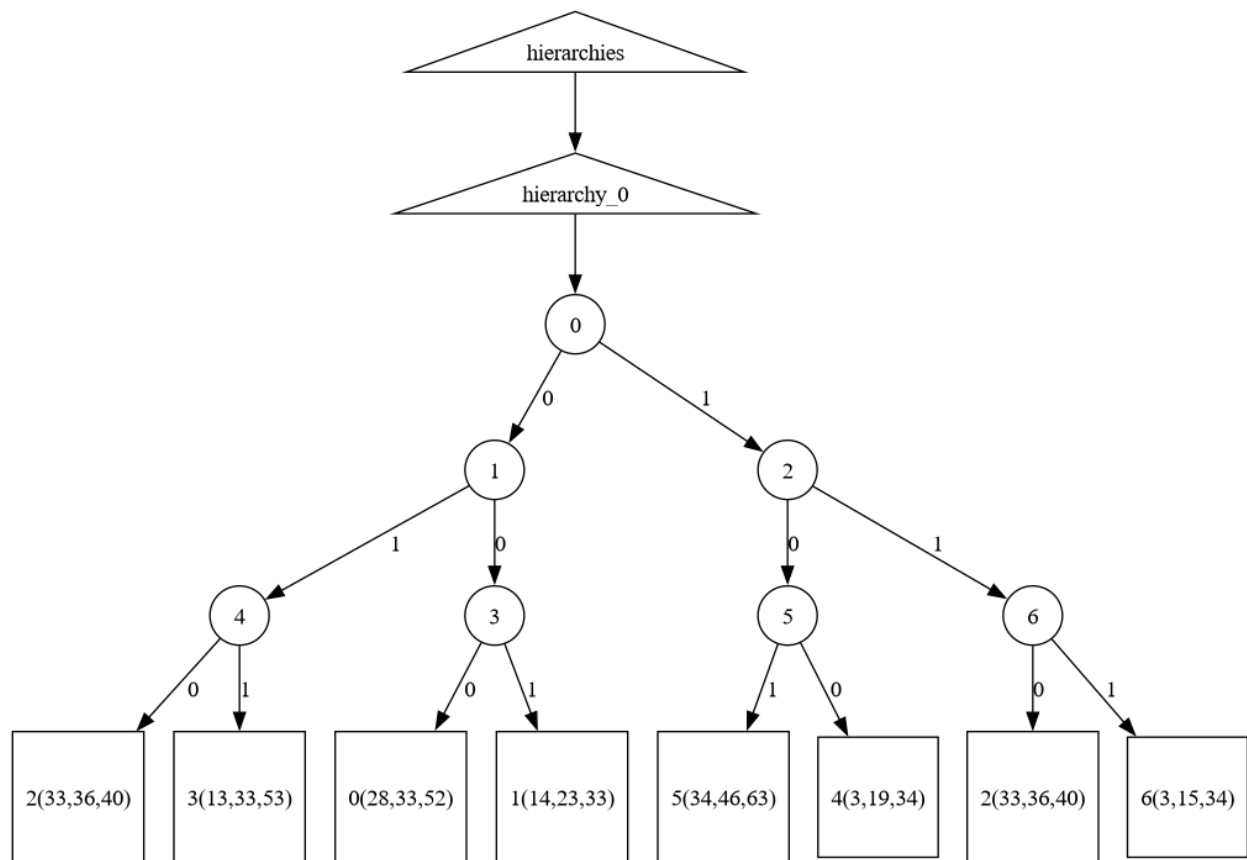


Figure 2 – Simple generated hierarchy

Here there are seven non-terminals (0-6), indicated by circles, and seven terminals, indicated by squares. The numbers on the edges indicate the child order. The parenthesized numbers in the terminals are sparse encodings for the neural network, to be discussed later.

This hierarchy produces a path of: {0, 1, 2, 3, 4, 5, 2, 6}.

The terminal children of a non-terminal are considered to be a cause-and-effect pair. The goal of the neural network is to predict the effect given the cause as they are presented in the input stream.

Generating inputs using this method presents context-related challenges. For example, the appearance of cause 2 cannot solely be used to determine which of the effects 3 or 6 is correct. The correct prediction depends on the context provided by the previous cause-and-effect pair.

Parameters determine the shape of a hierarchy:

- NUM_CAUSATION_HIERARCHIES
- NUM_NONTERMINALS

- NUM_TERMINALS
- TERMINAL_PRODUCTION_PROBABILITY

Multiple paths are generated by multiple hierarchies. Node ids are scoped by hierarchy, so the same id in different hierarchies will be unique to each hierarchy. The terminal production probability affects the hierarchy height: a greater probability causes non-terminals to be less likely to produce non-terminal children, thus growing the hierarchy further.

An example of two hierarchies having identical structure is presented in Figure 3.

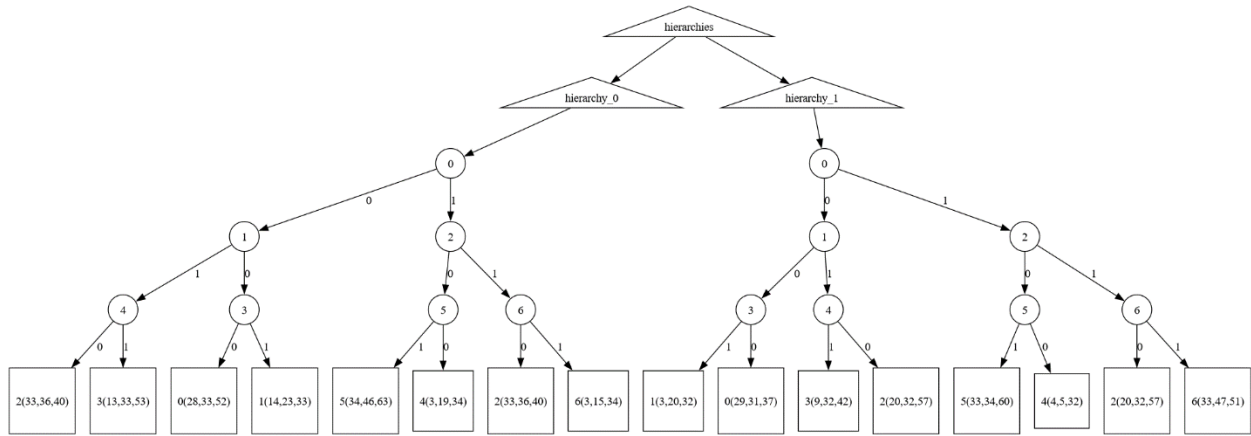


Figure 3 – Two hierarchies

Note that a cause terminal is not caused, which means that temporal gaps containing interstitial events can precede it, presenting another learning challenge. For example, with interstitial events the stream given by the hierarchy in Figure 1 might look like:

{a, b, 0, 1, b, 2, 3, b, c, 4, 5, 2, 6}

Where the letters a-c represent interstitial events.

A final challenge, addressing multitasking, can be shown by interleaving the streams from the hierarchies in Figure 2:

{0/0, 1/0, 2/0, 3/0, 0/1, 1/1, 4/0, 5/0, 2/1, 3/1, 2/0, 6/0, 4/1, 5/1, 2/1, 6/1}

Where the hierarchy number is appended to the terminal id after a slash ("/") character.

ALGORITHM

The generated terminal ids are encoded into a sparse representation of one-hot inputs to the neural network. Sparse encoding is also used to represent higher tiered information. The advantage of sparsity in neural networks is delineated in the Numenta White Paper (2021).

During training, paths are presented to the network without interstitial or interleaved events. Those are supplied during testing. This is how training is done:

The input to the network at time t is denoted X_t . The output is y_t . Concatenated to the external input and prediction output, there are n tiers of encoded temporal states.

The tier values for y_t are computed as follows:

```
update_y_tiers:
    tier_A = encode(external input)
    for tier i from 0 to n:
        tier_B =  $X_t$  values of tier i
        if tier_B exists:
            negate values for tier_B in tier i
            tier_A = encode(tier_A, tier_B)
        else:
            enter values for tier_A in tier i
            break
```

The tier values of $X_{t+1} = X_t + y_t$

The network is thus trained to compute temporal context information. Note that the y values are a delta for the X values of the current hierarchy, leaving the X values generated by other concurrent hierarchies to propagate unmodified.

Here is an example scenario, where the number triples are encodings:

```
t = 0, X: [sense=[4, 14, 41]], y: [sense=[21, 24, 40], tier0=[4, 14, 41]]
t = 1, X: [sense=[21, 24, 40], tier0=[4, 14, 41]], y: [sense=[5, 42, 43], tier0=[-4, -14, -41], tier1=[8, 9, 34]]
t = 2, X: [sense=[5, 42, 43], tier0=[], tier1=[8, 9, 34]], y: [sense=[38, 40, 42], tier0=[5, 42, 43]]
```

The algorithm also times-out values to limit their “range”. Values in higher tiers, having longer histories, persist longer.

For testing, novel interstitial and interleaved events are presented to the network to evaluate resilience to interruptions and multitasking performance. The prediction output is used to update the context tiers rather than the y values.

Note that the algorithm could also be used to recognize out-of-order streams, a topic explored by Portegys (2024a).

The code is available on GitHub at: <https://github.com/morphognosis/Mandala>

RESULTS

MANDALA/RNN COMPARISONS

Mandala was evaluated against an LSTM (Long Short-Term Memory) RNN using the Keras 3.11.3 machine learning library.

Mandala neural network parameters:

- Three hidden layers of 128 neurons.
- Tanh activation.
- Mean squared error loss.
- Adam optimizer.

RNN parameters:

- One hidden layer of 128 neurons.
- Mean squared error loss.
- Adam optimizer.

Training for 500 epochs. Average of 10 runs per parameter setting. An error was counted when the effect terminal event was not correctly predicted by its cause terminal event.

To get a baseline comparison, Mandala and the RNN were run with no interstitial or interleaved events. Figure 4 shows nearly perfect performance for both.

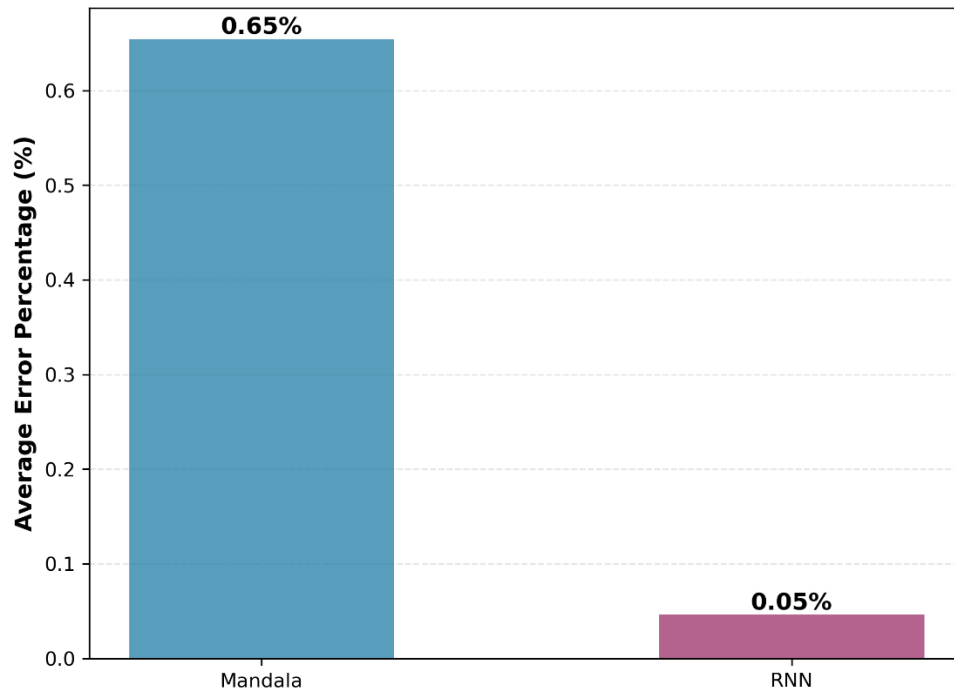


Figure 4 – Baseline testing performance: no interstitial or interleaved events

The remainder of the tests were done with interstitial events inserted into the stream, randomly chosen from 10 possible patterns.

Parameter combinations are as follows:

- NUM_CAUSATION_HIERARCHIES: 1, 2, 3.
- NUM_NONTERMINALS: 10, 15, 20.
- NUM_TERMINALS: 10, 15, 20.
- TERMINAL_PRODUCTION_PROBABILITY: .25, .5, .75

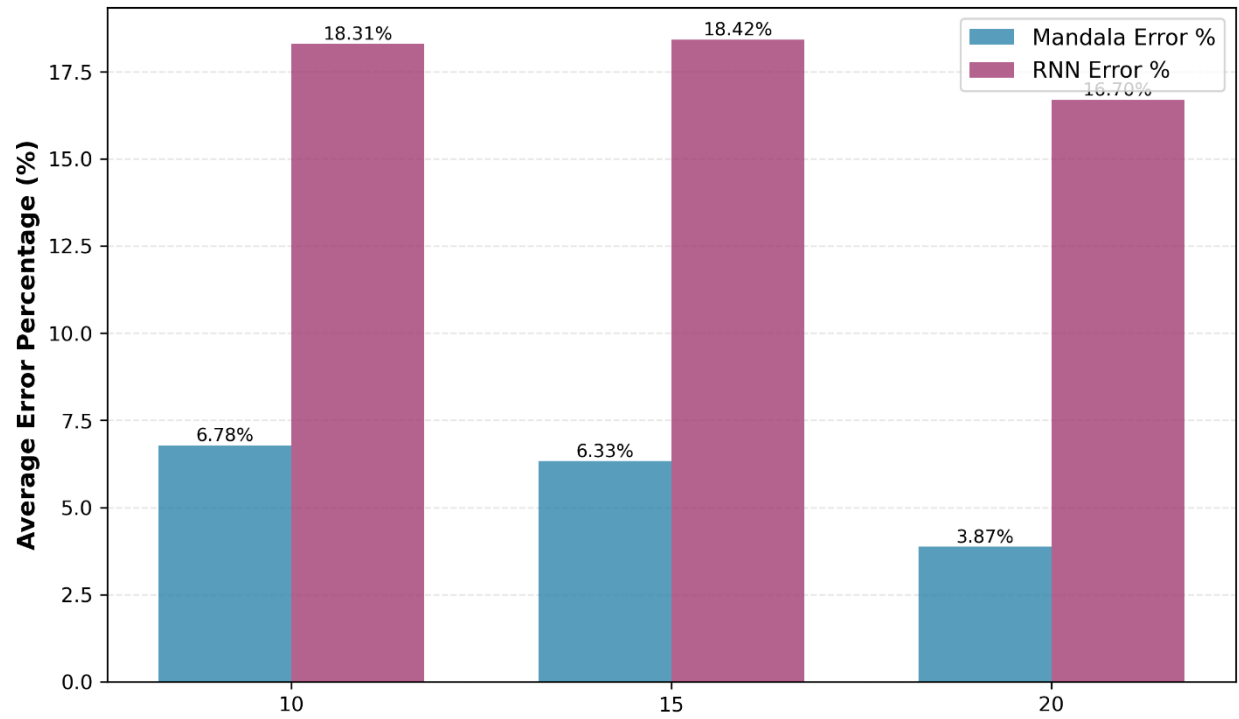


Figure 5 – Number of terminals

Figure 5 shows the results of varying the number of possible terminals.

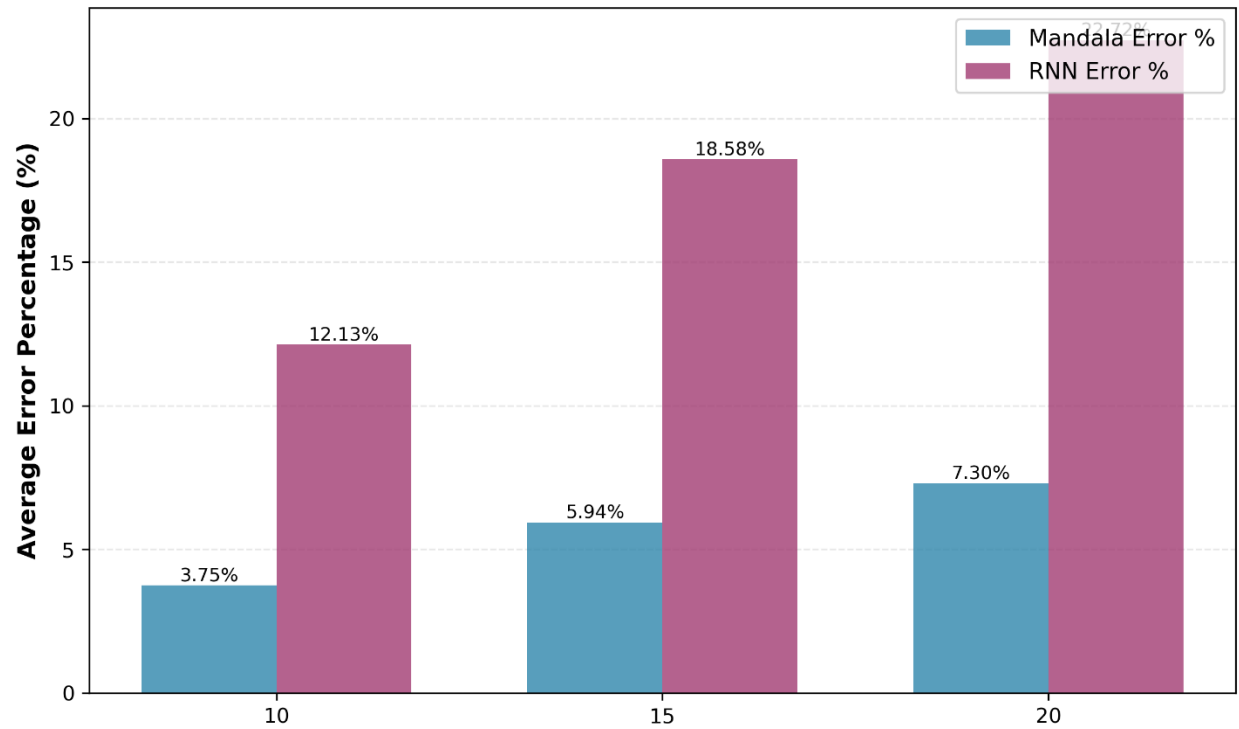


Figure 6 – Number of non-terminals

Figure 6 varies the number of non-terminals, which appears to indicate a linear-like increase in the error rate.

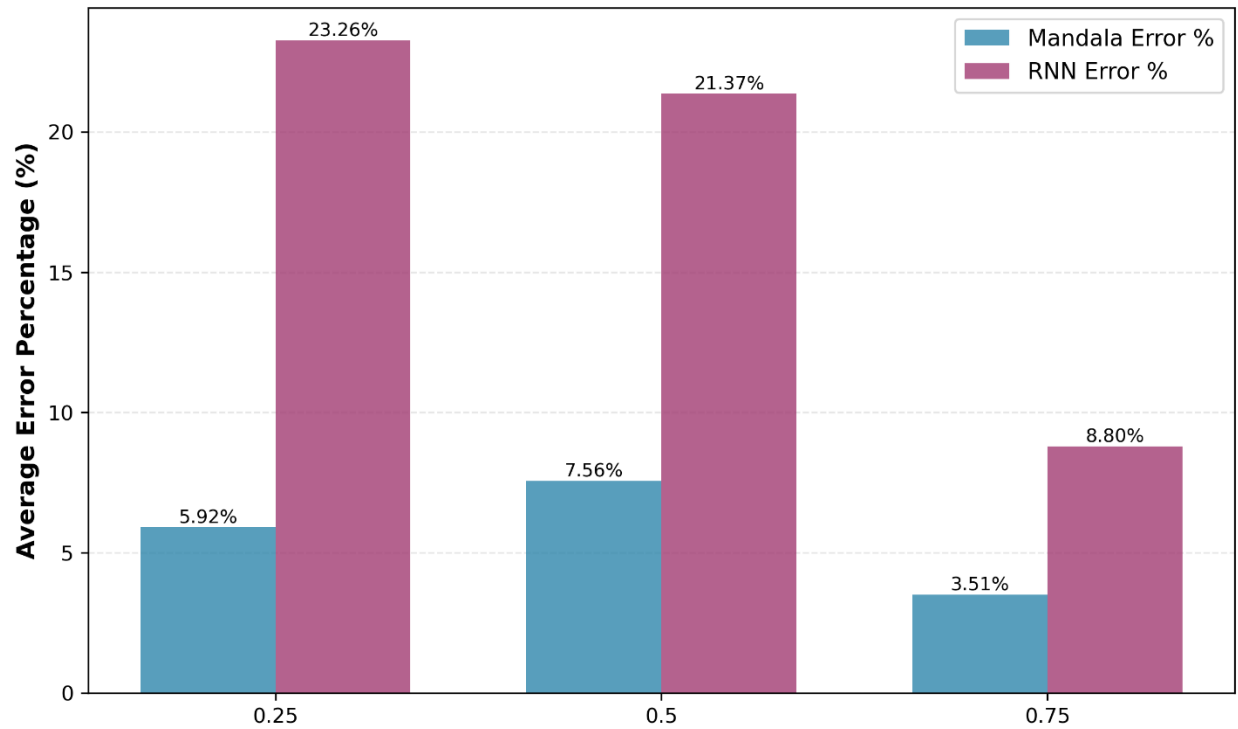


Figure 7 – Terminal production probability

Figure 7 shows how making the hierarchies shallower by increasing the probability of terminating branches reduces the error rate for both Mandala and RNN.

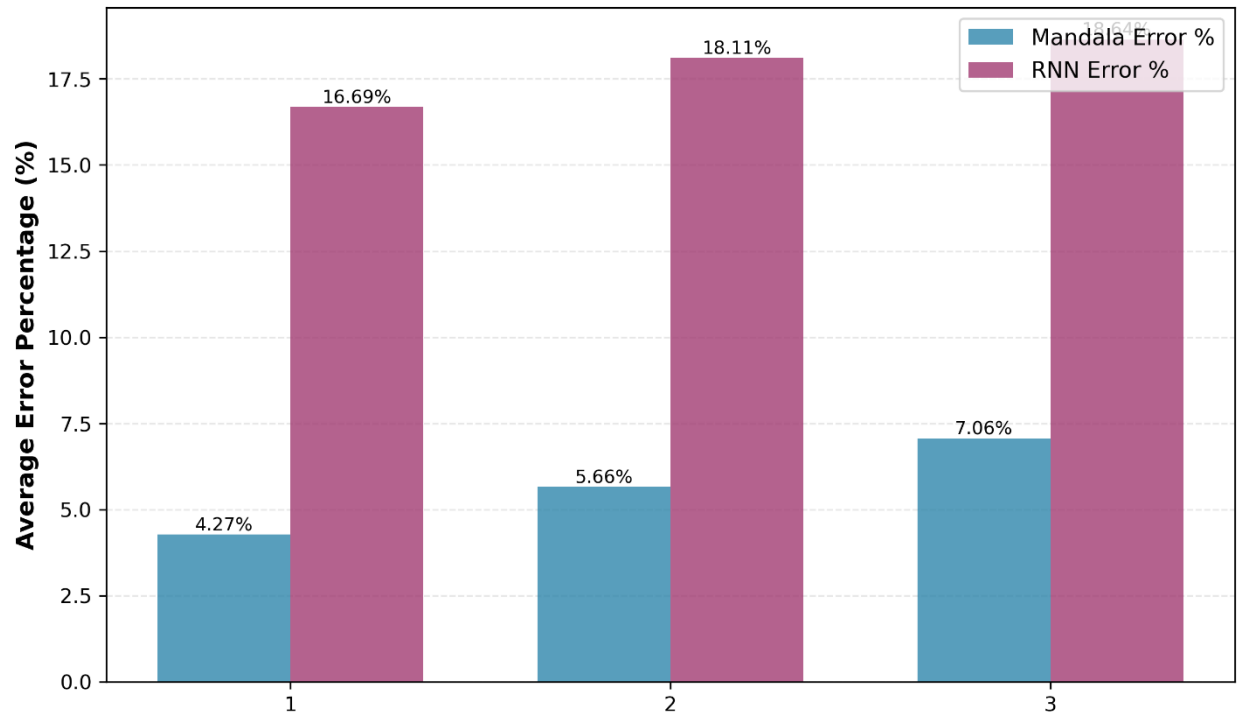


Figure 8 – Causation hierarchies

Figure 8 demonstrates how multitasking, which interleaves events, affects performance. The error rate for Mandala increases as the number of tasks goes from 1 to 3. RNN shows worse performance for multitasking in general.

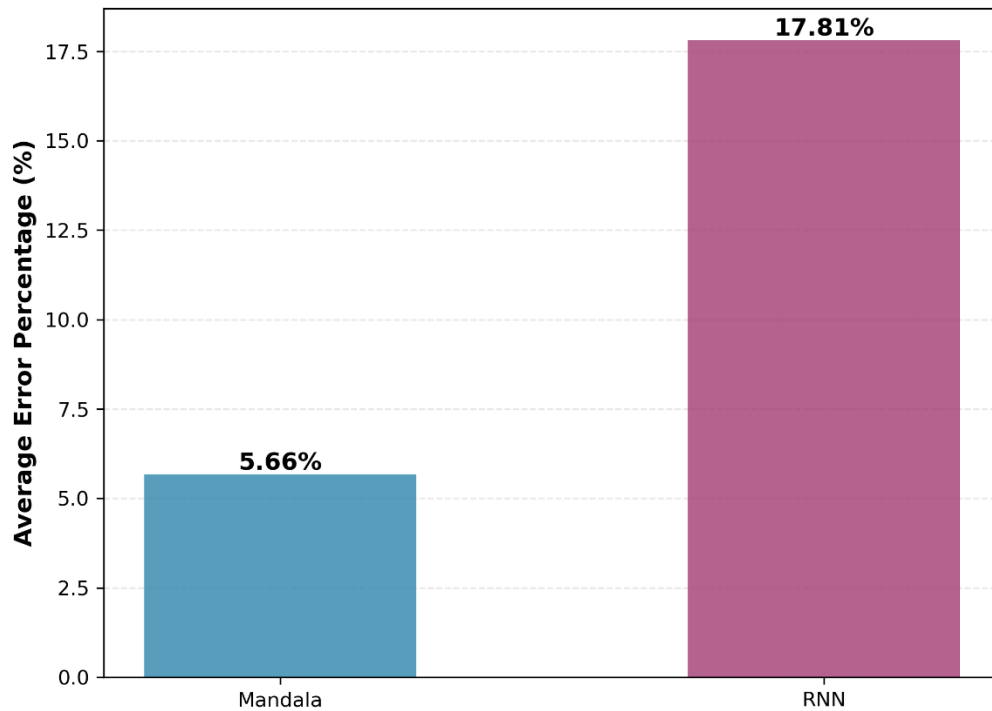


Figure 9 – Overall error rates

Figure 9 shows the overall error rate comparison across parameter combinations.

LLM TESTING

As an attempt to observe how an LLM performs on this type of problem, the sequences depicted in Figure 2 were posed to Claude, a popular LLM.

Prompt:

Given these sequences:

0,1,2,3

4,5,2,6

Predict the next number in these sequences:

8,8,7,0

7,11,4

Claude correct predicts 1 and 5 respectively.

Prompt:

Given these sequences:

0,1,2,3

4,5,2,6

Predict the next number in these sequences:

7,4,5,11,11,8,10,10,10,11,2

0,1,9,11,10,2

7,9,11,9,7,8,4,5,7,10,10,8,7,2

7,0,1,11,10,7,7,10,2

4,5,10,8,8,2

11,8,0,1,8,10,2

11,7,7,11,11,0,1,11,7,11,11,9,11,10,7,2

10,8,9,7,4,5,9,8,8,10,11,11,7,9,9,11,2

Note that 3 is the correct prediction if 4,5 is in the sequence, and 6 if 0,1 is in the sequence.

Claude ran a number of analytics, but failed to predict any of the sequences.

CONCLUSION

Mandala suggests that external hierarchical temporal state accumulation enables neural networks to handle interruptions and multitasking more effectively than conventional RNNs as evidenced by a 68% performance improvement over LSTMs (5.66% vs. 17.81% error rate).

By maintaining separable contextual tiers, Mandala preserves information about concurrent causation chains that might otherwise interfere in standard recurrent architectures. This approach offers both predictive capability and explicit causation labeling, making it particularly suitable for goal-directed systems like Mona that require identified causal relationships. A likely candidate for future work would be to retrofit the honey bee foraging system presented in Portegys (2022) to use Mandala.

REFERENCES

Forgione, M. (2020). How a pair of bald eagles became Southern California rock stars. *The Los Angeles Times*.

Graves, A., Wayne, G., & Danihelka, I. (2014). Neural Turing Machines. <https://arxiv.org/abs/1410.5401>.

Jordan, M. I. (1986). Serial order: a parallel distributed processing approach. Tech. rep. ICS 8604. San Diego, California: Institute for Cognitive Science, University of California.

Kumar, A. Irsoy, O., Ondruska, P., Iyyer, M., Bradbury, J., Gulrajani, I., Zhong, V., Paulus, R., Socher, R. (2016). Ask Me Anything: Dynamic Memory Networks for Natural Language Processing. *Proceedings of the 33rd International Conference on Machine Learning, PMLR* 48:1378-1387.

Numenta (2021). Sparsity Enables 100x Performance Acceleration in Deep Learning Networks, A Technology Demonstration. Numenta Whitepaper. <https://www.numenta.com/assets/pdf/research-publications/papers/Sparsity-Enables-100x-Performance-Acceleration-Deep-Learning-Networks.pdf>

Portegys, T. (2010). A Maze Learning Comparison of Elman, Long Short-Term Memory, and Mona Neural Networks. *Neural Networks*.

Portegys, T. (2022). Morphognostic honey bees communicating nectar location through dance movements. *Nature and Biologically Inspired Computing (NaBIC 2022)*.

Portegys, T. (2024a). Learning causation event conjunction sequences. *Journal of Artificial Intelligence and Autonomous Intelligence*.

Portegys, T. (2024b). Nest-building using place cells for spatial navigation in an artificial neural network", *ICALRAI 2024: International Conference on Artificial Life, Robotics and Artificial Intelligence*.

Quan, Z., Zeng, W., Li, X., Liu, Y., Yu, Y., Yang, W. (2020). Recurrent Neural Networks With External Addressable Long-Term and Working Memory for Learning Long-Term Dependences. *IEEE Transactions on Neural Networks and Learning Systems* 2020 Mar;31(3):813-826. doi: 10.1109/TNNLS.2019.2910302.

Yin, S., Fu, C., Zhao, S., Li, K., Sun, X., Xu, T., Chen, E. (2024). A Survey on Multimodal Large Language Models. *National Science Review*. 11 (12) nwae403. doi:10.1093/nsr/nwae403.